

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

по дисциплине «Естественно-языковой интерфейс интеллектуальных
систем»

на тему «Автоматическое реферирование текстовых документов
(вариант 16)»

Выполнили студенты гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Крапивин Ю. Б.

Минск 2026

Цель: изучить и отработать практические навыки применения методов автоматического реферирования текстовых документов.

Задание (вариант 16): разработать систему автоматического реферирования текстовых документов со следующей комбинацией признаков варианта:

- **Вид реферата** – классический реферат (выборка наиболее значимых предложений документа) и реферат в виде иерархического списка ключевых слов и словосочетаний;

- **Реализуемые методы отбора предложений** – алгоритм из методички (модифицированный TF-IDF с учётом положения предложения в документе и абзаце), TextRank и метод на эмбедингах предложений;

- **Улучшение реферата** – перефразирование отобранных предложений с помощью LLM без добавления новых фактов.

Согласно методическим указаниям, для каждого метода отбора предложений необходимо:

- а) рассчитать вес каждого предложения документа в соответствии с правилом метода;

- б) отобрать заданное число предложений с наибольшим весом, сохранив их исходный порядок следования в документе;

- в) отдельно построить реферат-список ключевых слов: термины документа с наибольшим весом, при возможности сгруппированные в иерархию (корневое слово – словосочетания, в которые оно входит), например: лазер – красный лазер, синий лазер;

- г) сравнить результаты разных методов по числу отобранных предложений, степени сжатия и затраченному времени.

Кроме того, система должна:

- на выходе выдавать активную ссылку на документ, текст реферата (классического и в виде ключевых слов) и сводную статистику сравнения методов по всей тестовой коллекции;

- предоставлять средства сохранения результата в файл и печати;

- иметь простой и доступный пользовательский интерфейс со справочными подсказками.

Структура разработанной системы

Реферирование документов реализовано как новый сценарий использования поверх архитектуры ЛР1–ЛР2 (см. docs/ARCHITECTURE.md): границы сервисов по-прежнему проведены по зоне ответственности, а не по лабораторной работе, поэтому ЛР3 добавляет один новый сервис и новые таблицы

БД, не меняя ни одного существующего контракта системы. Структура системы на уровне контейнеров (C4, уровень 2) показана на рисунке 1; компоненты и связи, добавленные именно в этой лабораторной работе, выделены зелёным цветом.

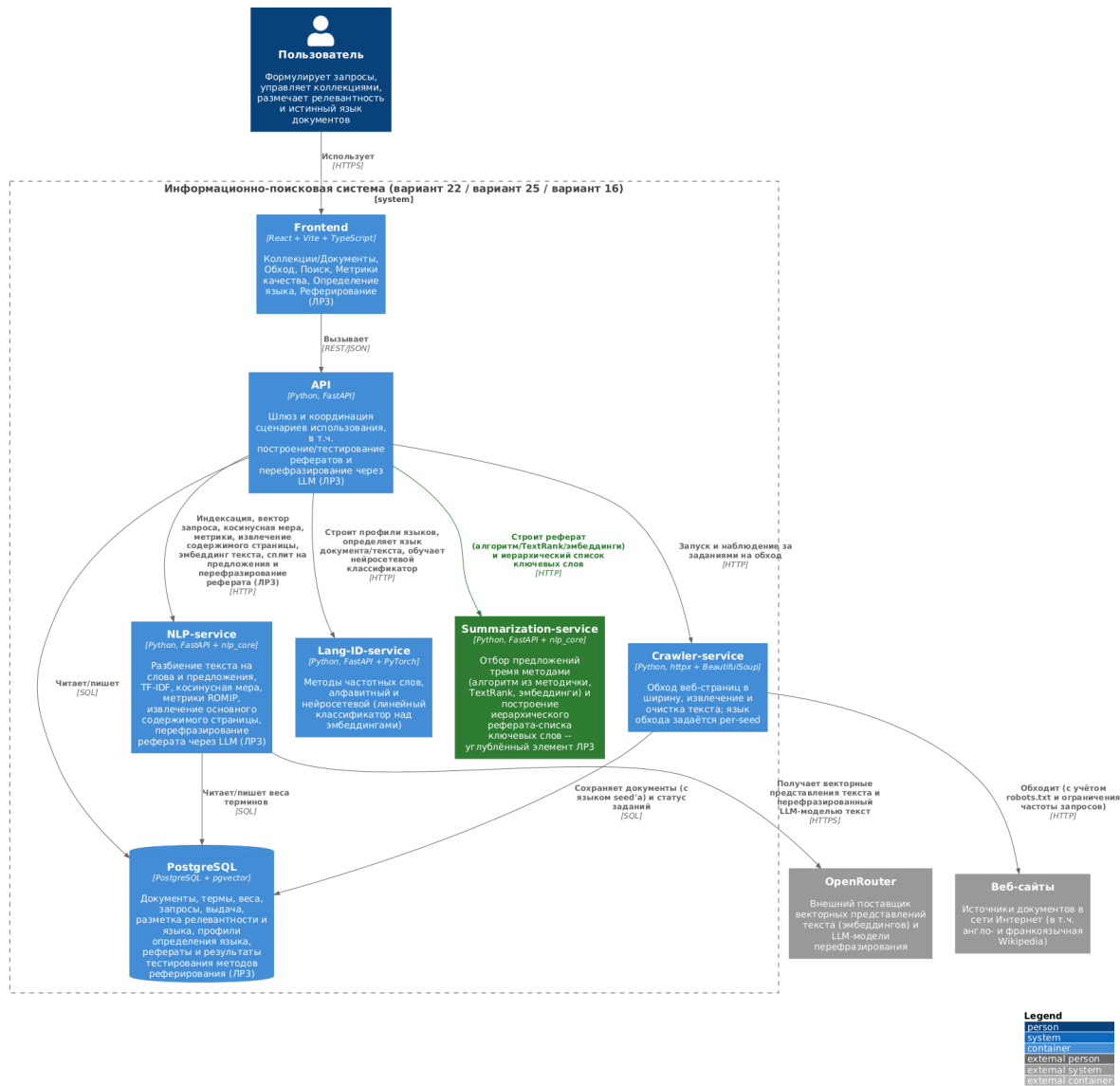


Рисунок 1 – Диаграмма контейнеров разработанной системы (нотация C4); зелёным – новое в ЛРЗ

Новый сервис **summarization-service** (Python, FastAPI + **nlp_core**) – углублённый элемент варианта 16 (реализация методов реферирования). Он, как и **nlp-service** и **lang-id-service**, не хранит состояние и не обращается к БД самостоятельно: получает на вход уже подготовленные данные (текст, веса терминов документа, эмбединги предложений) и возвращает чистый результат вычисления – отобранные предложения с их весами либо иерархический список ключевых слов. Оркестрацией (какие документы проиндексированы, что сохранить в БД, что показать пользователю) занима-

ется `api`, как и для остальных сценариев системы. Веса терминов документа (модифицированный TF-IDF) для алгоритмического метода считаются в `api` тем же кодом, что и для ключевых слов – переиспользуется формула `LP1 modified_term_weight`. Помимо нового сервиса, ЛРЗ расширила ответственность `frontend`:

- новая страница «Реферирование» с двумя вкладками (реферат документа, тестирование на коллекции);

- поиск документа реализован не выпадающим списком со всеми документами коллекции, а комбобоксом с поиском на сервере (`DocumentCombobox`) – по мере набора текста подгружается только страница из не более чем 30 совпадающих по названию документов (аналогично добавлен параметр `search` в `GET /collections/{id}/documents`, реализованный через `ILIKE` с экранированием служебных символов `LIKE`). Так список остаётся быстрым и при выборе из коллекции на 20 документов, и из коллекции на несколько тысяч – в отличие от предыдущего варианта, подгружавшего до 500 документов целиком в один выпадающий список;

- кнопка «Печать» открывает не системную печать всей страницы приложения, а отдельную вкладку с `Blob URL`, содержащую только текст реферата, иерархию ключевых слов и (если оно было построено) перефразированную `LLM` версию – без интерфейса приложения; результат сохраняется в файл в том же составе.

Общая схема построения реферата одного документа показана на рисунке 3; методы отличаются только правилом расчёта веса предложения (шаг 2), реферат-список ключевых слов (шаг 4) строится один раз и не зависит от метода отбора предложений.

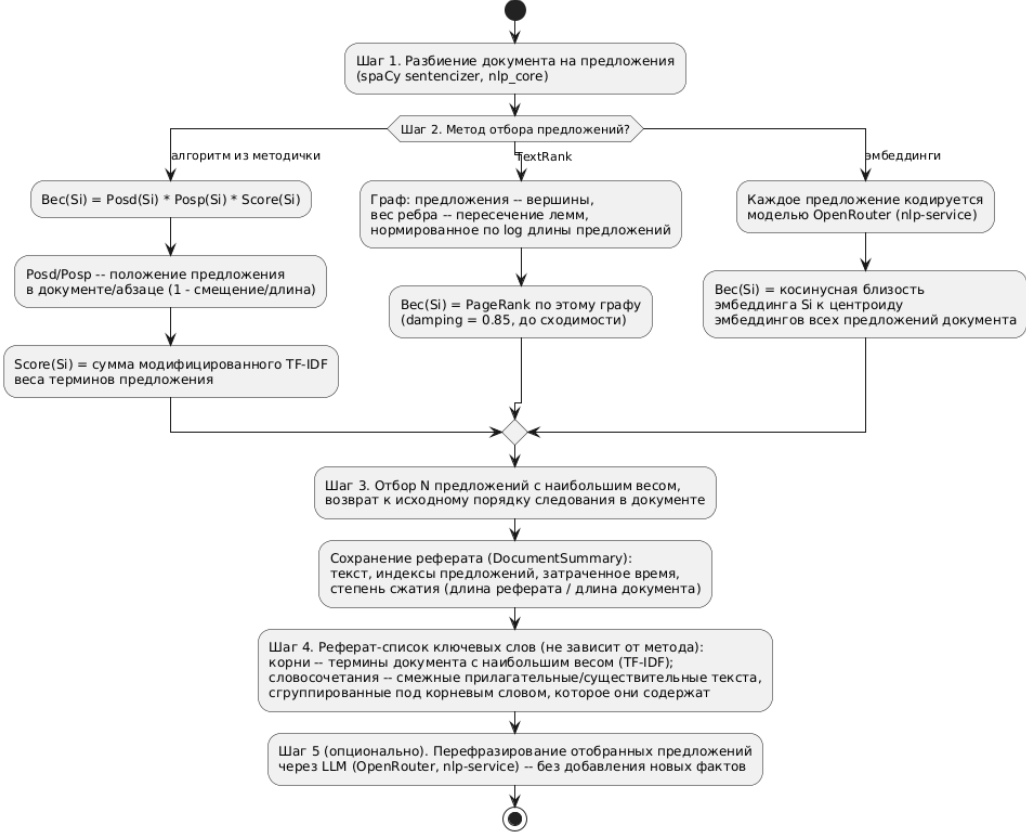


Рисунок 3 – Построение реферата одного документа (общая схема для всех трёх методов)

Алгоритм из методички. Вес предложения – произведение трёх функций:

$$\text{Bec}(S_i) = \text{Posd}(S_i) \cdot \text{Posp}(S_i) \cdot \text{Score}(S_i), \quad (1)$$

где $\text{Posd}(S_i) = 1 - BD(S_i)/|D|$ и $\text{Posp}(S_i) = 1 - BP(S_i)/|P|$ – положение предложения в документе и в абзаце (предложения ближе к началу весят больше), а $\text{Score}(S_i) = \sum_{t \in S_i} tf(t, S_i) \cdot w(t, D)$ – сумма модифицированного TF-IDF веса терминов предложения (та же формула $w(t, D) = 0,5 \cdot (1 + tf/tf_{\max}) \cdot \log(|DB|/df(t))$, что и в ЛР1).

TextRank. Графовый метод: предложения – вершины, вес ребра между S_i и S_j – пересечение множества лемм, нормированное по длине предложений:

$$w_{ij} = \frac{|W_i \cap W_j|}{\log(|W_i| + 1) + \log(|W_j| + 1)}, \quad (2)$$

итоговый вес предложения – результат PageRank по этому графу ($\text{damping} = 0,85$, до сходимости или 100 итераций). Не требует индексации коллекции и работает по каждому документу независимо.

Эмбединги. Каждое предложение кодируется моделью OpenRouter (та же модель, что и для семантического поиска в ЛР1); вес предложения – косинусная близость его эмбединга к центроиду эмбедингов всех предложений документа. Для всех трёх методов итоговый реферат – N предложений с наибольшим весом, возвращённые к исходному порядку следования в документе.

Реферат-список ключевых слов. Корневые термины – весь словарь документа (все термины с ненулевым модифицированным TF-IDF весом), отсортированный по убыванию веса, без искусственного ограничения сверху (общий для всех трёх методов расчёт, не зависит от способа отбора предложений) – для коротких тестовых текстов это означает, что дерево ключевых слов покрывает всю лексику документа, а не только первые 15 самых весомых терминов, как было в первой версии реализации. Словосочетания-дети строятся без синтаксического парсера: последовательные прилагательные/существительные текста (POS-теги spaCy) длиной ≥ 2 слов – кандидаты в словосочетания, каждое оценивается суммой весов входящих в него терминов и закрепляется за тем корнем с наибольшим весом среди уже незанятых корней, который оно содержит (не более 5 словосочетаний на корень). Корень без подходящих словосочетаний остаётся листом дерева – так реферат остаётся корректным и для документов, где нет устойчивых словосочетаний вокруг темы. Пример результата – на рисунке 4.

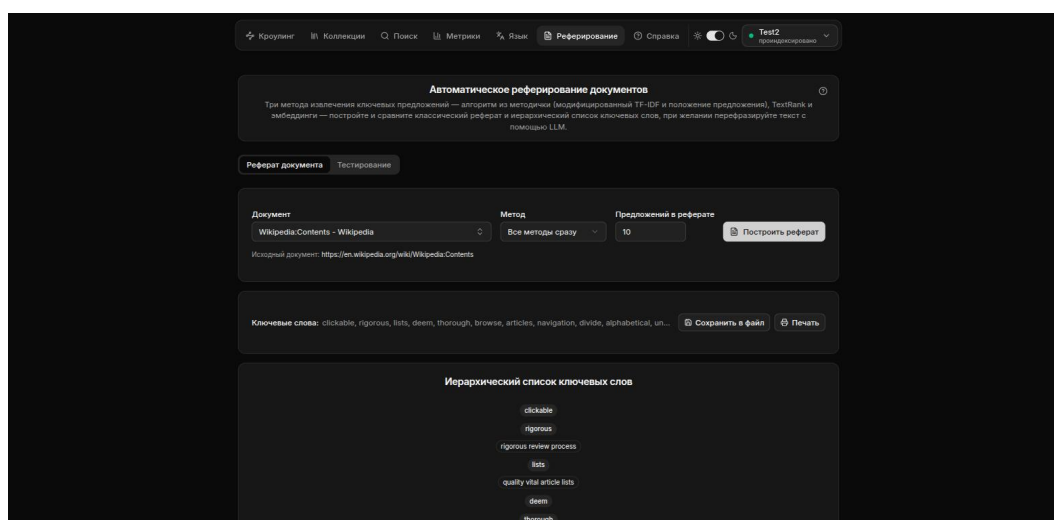


Рисунок 4 – Вкладка «Реферат документа»: реферат-список ключевых слов в виде дерева (корень – словосочетания, в которые он входит)

При выборе метода «Все методы сразу» рефераты всех трёх методов строятся одним запросом и выводятся друг под другом (не в колонках) –

рисунок 5; перефразированная LLM версия реферата появляется прямо под тем методом, к которому она относится (рисунок 6).

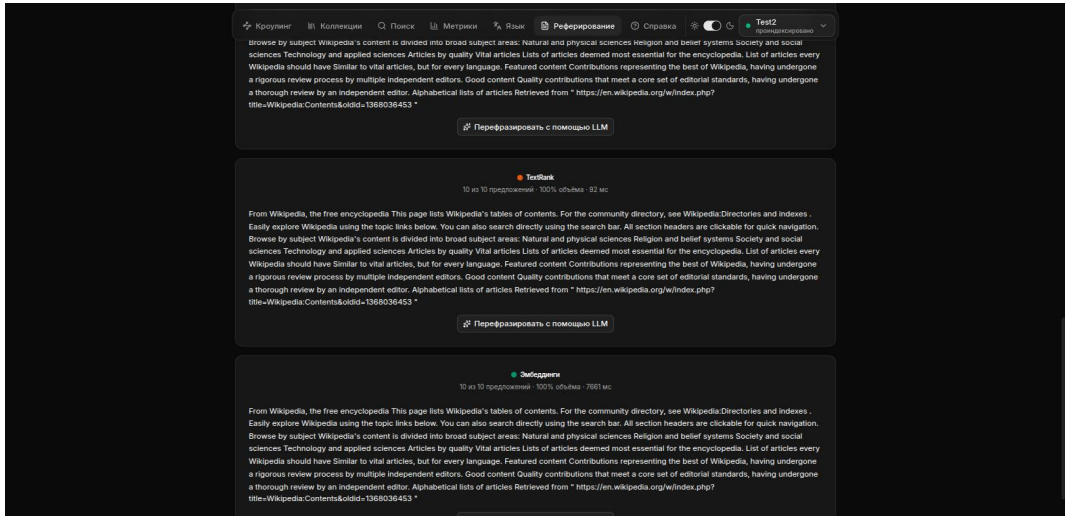


Рисунок 5 – Результаты всех трёх методов, построенные одним запросом («Все методы сразу») и выведенные друг под другом

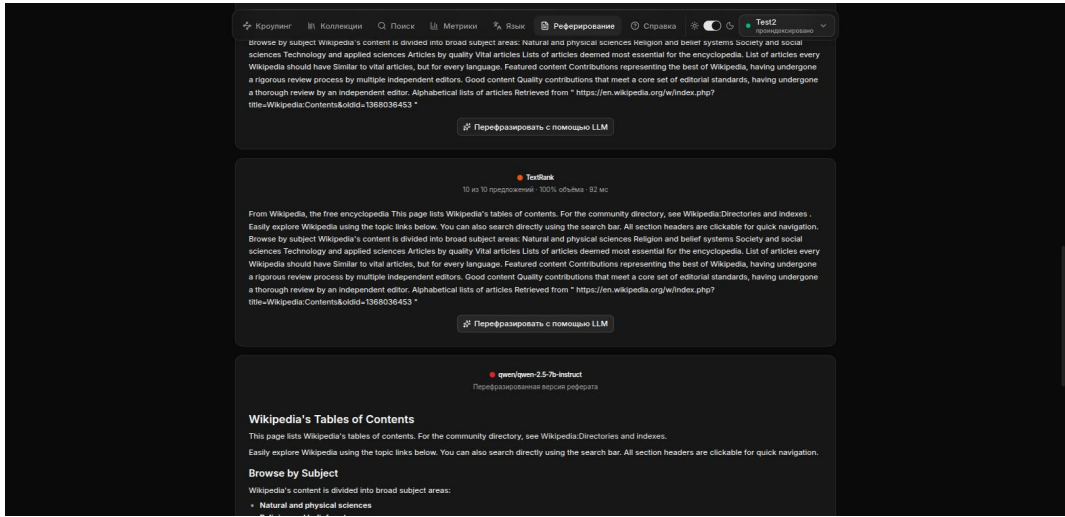


Рисунок 6 – Перефразированная с помощью LLM версия реферата под соответствующим методом

Перефразирование через LLM. Запрос к модели состоит из системного промпта (листинг 1, задаёт единственную задачу – лёгкую правку уже отобранных предложений без добавления, удаления или дальнейшего сокращения информации) и пользовательского сообщения, равного тексту реферата (с необязательной пометкой языка первой строкой, если он известен из карточки документа). Такой узкий промпт – единственная защита от типичной проблемы LLM в задаче реферирования: модель дописывает отсутствующие в источнике факты или самостоятельно сокращает текст ещё раз, из-за чего результат перестаёт быть точным отражением уже отобранных алгоритмом предложений.

```
1 You are a copy-editing assistant for an extractive text-summarization tool.
  ↳ You are given sentences that an algorithm already selected as the most
  ↳ important ones from a source document. Your ONLY task is to lightly
  ↳ polish them into a clean, coherent passage: fix grammar, smooth the
  ↳ transitions between sentences, and resolve obviously dangling
  ↳ pronouns/references when the antecedent is already present in the given
  ↳ text.
2
3 Strict rules:
4 1. Do NOT add any fact, number, claim, or detail that is not already present
  ↳ in the input sentences.
5 2. Do NOT omit or change the meaning of any of the given sentences.
6 3. Do NOT summarize further -- keep all the given information.
7 4. Write in the same language as the input text.
8 5. Respond in Markdown: plain paragraphs are enough, only use headings,
  ↳ lists or emphasis if they genuinely improve readability of this exact
  ↳ content.
```

Листинг 1 – Системный промпт для LLM-полировки реферата
(services/nlp-service/app/llm.py)

Тестовая коллекция документов

Тестовая коллекция – та же коллекция англоязычных статей Wikipedia, наполненная краулером ЛР1 (глубина 3, переход только в пределах домена en.wikipedia.org), что используется для проверки поиска: 20 проиндексированных документов, на которых алгоритмический метод может рассчитать модифицированный TF-IDF (без индексации коллекции доступны только TextRank и метод на эмбедингах, не требующие весов терминов). Число предложений в реферате – 10 (по умолчанию, как рекомендует методичка); фактическое среднее число отобранных предложений – 7,3, т.к. часть документов короче 10 предложений и для них отбираются все доступные.

Результаты тестирования и оценка качества

Каждый из трёх методов протестирован на всех 20 документах коллекции; средние по документам метрики приведены в таблице 1, тот же расчёт в интерфейсе системы – на рисунке 7 (вкладка «Тестирование»).

Таблица 1 – Метрики качества и быстродействия по 20 документам тестовой коллекции

Метрика	Алгоритм из методички	TextRank	Эмбединги
Среднее время реферирования	534,26 мс	289,45 мс	4013,54 мс
Средняя степень сжатия	70,4 %	69,6 %	65,9 %
Среднее число предложений в реферате	7,3	7,3	7,3
Документов обработано	20	20	20

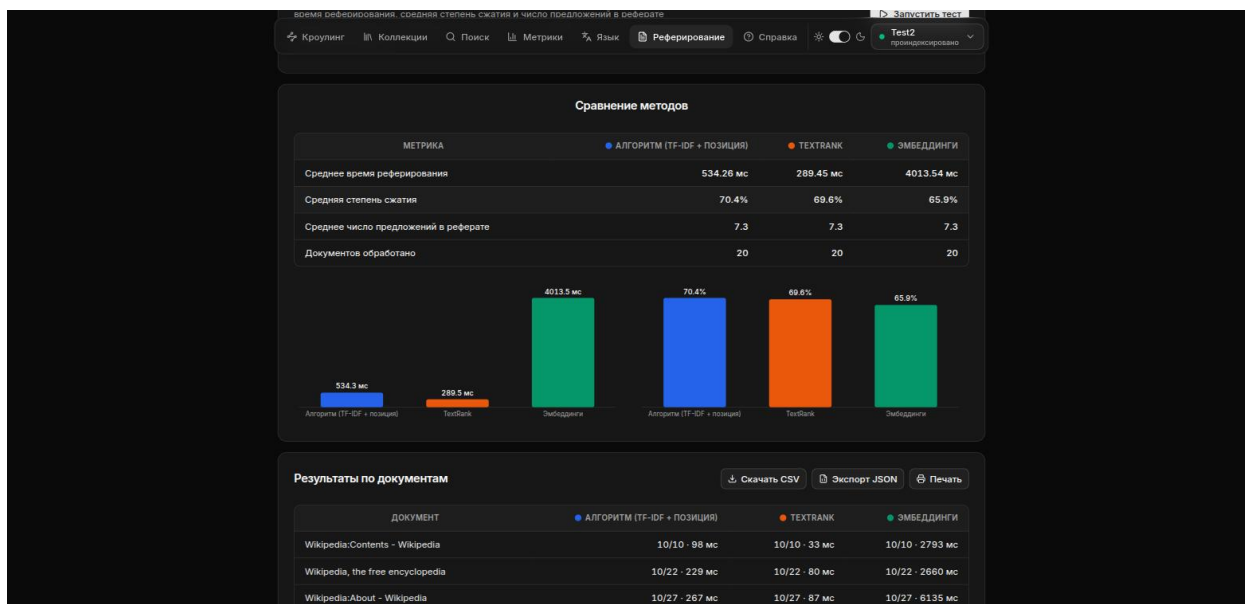


Рисунок 7 – Вкладка «Тестирование»: сравнение методов по времени и степени сжатия, результаты по документам с экспортом (CSV/JSON) и печатью

Степень сжатия считается по длине текста в символах, а не по числу предложений. Изначально степень сжатия считалась как отношение числа отобранных предложений к общему числу предложений документа. Все три метода при сравнении на одной коллекции получают на вход один и тот же `sentence_count` и работают с одним и тем же документом (тем же числом предложений), поэтому такое отношение математически гарантированно совпадает у всех трёх методов – оно вообще не зависит от того, какие именно предложения выбрал метод, а только от того, сколько предложений было запрошено. Как метрика сравнения методов она была бесполезна по построению, а не только случайно совпадала на конкретных данных. Метрику пересчитали на длину извлечённого текста в символах относительно длины документа (`total_chars`, таблица 1) – эта величина действительно зависит от того, какие (более короткие или более длинные) предложения выбрал метод, и поэтому различает методы (70,4 / 69,6 / 65,9%, таблица 1).

Анализ результатов и предложения по улучшению

а) **TextRank – самый быстрый метод, эмбединги – самый медленный, разница на порядок.** Среднее время на документ – 289,45 мс у TextRank против 534,26 мс у алгоритма из методички и 4013,54 мс у метода на эмбедингах (табл. 1). TextRank работает целиком локально (лемматизация и граф предложений); алгоритмический метод дополнительно обращается к БД за весами терминов; метод на эмбедингах ждёт

HTTP-round-trip к OpenRouter на каждое предложение документа – сетевая задержка доминирует над самим вычислением косинусной близости.

б) **Алгоритм из методички даёт самую высокую степень сжатия, эмбединги – самую низкую (70,4 против 65,9 %).** Правдоподобное объяснение, вытекающее из самого правила отбора: Posd отдаёт предпочтение предложениям в начале документа, а первые предложения статей Wikipedia (как правило, развёрнутое вводное предложение-определение) в среднем длиннее рядовых; метод на эмбедингах ищет предложение, ближайшее к семантическому центроиду документа, а такие предложения чаще короче и конкретнее (единичный факт, а не общее вводное описание). TextRank – посередине, так как граф пересечения лемм не даёт систематического смещения ни к началу документа, ни к длине предложения.

в) **Что можно добавить в систему для её улучшения.**

- ограничение реферата по символам/словам, а не только по числу предложений – при сильно отличающейся длине предложений между документами текущий параметр (число предложений) даёт менее предсказуемый по факту итоговый объём текста, чем прямое ограничение длины;

- кеширование весов терминов документа (сейчас пересчитываются заново при каждом построении реферата и при каждом документе тестового прогона) – ускорило бы прежде всего алгоритмический метод при повторных запусках на одной коллекции;

- синтаксический парсер (dependency parsing) вместо текущего эвристического выделения словосочетаний по последовательным ADJ/NOUN-тегам – корректно бы разделял, например, однородные члены и не склеивал случайно соседние по тексту, но синтаксически не связанные слова в одно словосочетание;

- параллельные запросы эмбедингов предложений вместо последовательных – основная причина отставания метода на эмбедингах по скорости, ограничение сейчас только на стороне клиента nlp-service.

г) **Где систему можно применить.**

- построение краткой выжимки и облака тем для документов, найденных информационно-поисковой системой ЛР1 – перед открытием полного текста;

- предварительный просмотр содержимого перед разметкой истинного языка/релевантности в ЛР1–ЛР2 – реферат и ключевые слова быстрее прочитать, чем весь документ;

- учебный полигон для последующих лабораторных работ курса, использующих понятие значимого предложения/термина документа (например, генерация ответов на естественно-языковые запросы) – веса терминов и разбиение на предложения уже переиспользуются из ЛР1 и общей библиотеки nlp_core.

Использованные готовые компоненты

Таблица 2 – Библиотеки и программные средства, использованные при реализации ЛР3 (дополнительно к перечисленным в отчётах по ЛР1–ЛР2)

Компонент	Назначение
spracy (POS-теги, tok.pos_)	Выделение кандидатов в словосочетания для реферата-списка ключевых слов – те же теги, что уже использовались для лемматизации в ЛР1, без дополнительных моделей.
qwen/qwen-2.5-7b-instruct (через OpenRouter)	Перефразирование уже отобранных предложений – небольшая и дешёвая модель со строгим промптом (только грамматика и связность, без новых фактов), та же интеграция OpenRouter, что и для эмбедингов.
Qwen3-Embedding-8B (через OpenRouter)	Источник эмбедингов предложений для метода на эмбедингах – та же модель, что и в ЛР1/ЛР2; для реферата отдельного документа эмбединги считаются по предложениям (не по документу целиком, как в ЛР1).

Выводы

В ходе выполнения лабораторной работы реализована система автоматического реферирования текстовых документов по варианту 16 (классический реферат тремя методами – алгоритм из методички, TextRank, эмбединги – и реферат в виде иерархического списка ключевых слов, с перефразированием через LLM) как новый сценарий использования поверх архитектуры, разработанной в ЛР1–ЛР2: добавлен один новый сервис (**summarization-service**), три новые таблицы БД и новая страница интерфейса, при этом ни один существующий контракт системы (поиск, краулинг, определение языка) не был нарушен. Все три метода подчиняются единой схеме (расчёт веса предложения, отбор N предложений с наибольшим весом, возврат к исходному порядку), различаясь только правилом расчёта веса; реферат-список ключевых слов строится единообразно поверх весов терминов, уже посчитанных для отбора предложений, и не зависит от выбранного метода.

На тестовой коллекции из 20 документов англоязычной Wikipedia TextRank оказался почти в 14 раз быстрее метода на эмбедингах (289,45 мс против 4013,54 мс на документ, табл. 1) за счёт полностью локального

вычисления без обращений к внешнему API. Алгоритм из методички дал самую высокую степень сжатия (70,4 %), метод на эмбедингах – самую низкую (65,9 %) – разница объясняется систематическим смещением каждого метода к определённому типу предложений (первые/длинные предложения у Posd/Posp против семантически центральных/коротких у эмбедингов). Сама метрика степени сжатия в процессе работы над лабораторной была пересчитана с отношения числа предложений (гарантированно одинакового у всех методов при одинаковом **sentence_count** и потому непригодного для сравнения методов) на отношение длины текста в символах, которое действительно от метода зависит – показательный пример того, что метрика сравнения обязана быть чувствительна именно к тому, что отличает сравниваемые варианты. Разработанный интерфейс поддерживает как разовое построение реферата одним или сразу всеми методами (результаты выводятся друг под другом, а не в колонках), так и пакетное тестирование на всей коллекции, с выгрузкой результата в CSV/JSON и печатью отдельной страницы с содержимым (включая перефразированный LLM текст, если он был построен) – без интерфейса приложения, что закрывает соответствующее требование методических указаний. Отдельно, для практичности на реальных данных, список документов для выбора реализован не статичной загрузкой всей коллекции, а комбобоксом с поиском на сервере, что сохраняет работоспособность интерфейса при росте коллекции до тысяч документов.